DSDA 385 Homework 2 Report

Dylan Costello

March 2026

1 Introduction

The purpose of this assignment is to train two object detection models on two small datasets. Our goal is to see how AI models perform under limited computational resources and a dearth of data to train on. The models we will be training are a Faster R-CNN model and a YOLO model. And the datasets we will be using are the Penn-Fudan Pedestrian dataset and a subset of the Oxford-IIIT Pet dataset.

2 Steps to Reproduce Results

Step 1: Download all 10 python files and both of the data folders located in this link: Download Folder. Ensure all folders are extracted and placed inside on larger shared folder. **Step 2:** Open the terminal and navigate to the folder where all of the code is stored. **Step 3:** Type 'python main.py' and, as long as you have all the necessary packages installed, the program will run. **Note:** To find example images of the output of the models navigate to the 'runs' folder that will be created. In there should be several folders. The ones that matter are the 'qualitative' folder, as that will contain example predictions from the RCNN model. And the 'val' folder which will contain example predictions from the YOLO model.

3 Dataset Description

3.1 Penn-Fudan Pedestrian

This dataset consists of 174 images of pedestrians. There is only one class for the models to detect, that class being 'person.' All other objects are considered background and are not assigned a class. Bounding boxes are given for each image as well.

3.2 Oxford-IIIT Pet

This full dataset contains far too many images for our limited resources, so we created a subset of 10 breeds of cat for the model to analyze. Each image contains a picture of one cat and assigns a bounding box and class to each. The ten chosen cat breeds were "Abyssinian", "Bengal", "Birman", "Bombay", "British_Shorthair", "Egyptian_Mau", "Maine_Coon", "Persian", "Ragdoll", and "Siamese." This subset consists of 988 images.

4 Model Explanation

4.1 Faster R-CNN

Faster R-CNN is a two step object detection model. The first step proposes potential regions for the bounding boxes that might contain objects. The second step uses those proposed regions to predict the class of the object and refines the the bounding box to better reflect where the object is truly located. This is faster than a regular CNN that would go through each image pixel by pixel and classify each pixel individually. This model only looks at the boxes that it thinks will contain objects.

4.2 YOLOv8n

You Only Look Once, or YOLO, detection is an object detection model designed to only require an image to be passed through the model once. Unlike Faster R-CNN, which looks at each region individually, this model takes the image as an input, and then runs it through multiple layers of computing, before producing the output object classification while only looking at the image once.

5 Training Details

5.1 Faster R-CNN

The Faster R-CNN model was run on a MobileNet backbone with pretrained weights to ensure that the small dataset would not matter as much compared to if the model was trained from scratch. For both datasets the training batch size was 2, the validation batch size was 1, and the learning rate was 0.005. The small batch size was necessary due to 8GB GPU memory constraints. The number of epochs for the Oxford Pet dataset was 20 and for the Penn-Fudan dataset was 15.

5.2 YOLO

The YOLO model was imported from the python 'ultralytics' class. This allowed for pretrained weights to be imported and not require me to build the model from scratch. We specifically used the nano version of YOLO because it works more effectively on machines with fewer computational resources. For both datasets the training batch size was 8 and the learning rate was 0.001. The number of epochs for the Oxford Pet dataset was 20 and for the Penn-Fudan dataset was 15.

6 Results

6.1 Table

The following table was produced after running the models under the conditions described above:

Table 1: Final Test Metrics Summary

Dataset	Model	mAP50	mAP50_95	Precision	Recall	Inference Speed (images/sec)
Oxford Pet	RCNN	0.6342	–	0.6751	0.8926	31.6317
Oxford Pet	YOLO	0.8983	0.8272	0.7426	0.9071	4.2579
Penn-Fudan	RCNN	0.4919	–	0.9839	0.8714	28.6496
Penn-Fudan	YOLO	0.9726	0.7600	0.9680	0.8651	14.1651

Note: The mAP50_95 is automatically provided by YOLO although the assignment did not specifically require it.

Table 1 shows that YOLOv8n consistently outperformed Faster R-CNN, achieving higher mAP50 on both datasets. In the case of the Oxford Pet dataset, it also achieved higher precision and recall, while being only slightly worse in precision and recall with the Penn-Fudan dataset. The lower mAP50 for Faster R-CNN likely reflects both the small dataset size and the model's tendency to produce multiple boxes per object (Discussed later).

The biggest difference between these two was the inference speed, with RCNN performing much better than YOLO. However, this inference speed discrepancy may be misleading. I believe that the YOLO inference speed was calculated including the overhead time that it took to load the images into memory. So I believe the reported inference speed is actually higher for YOLO than what was reported. I have not verified this extensively, but using a personal timer and some math I believe these numbers are misleading and incorrect.

6.2 Images

6.2.1 RCNN

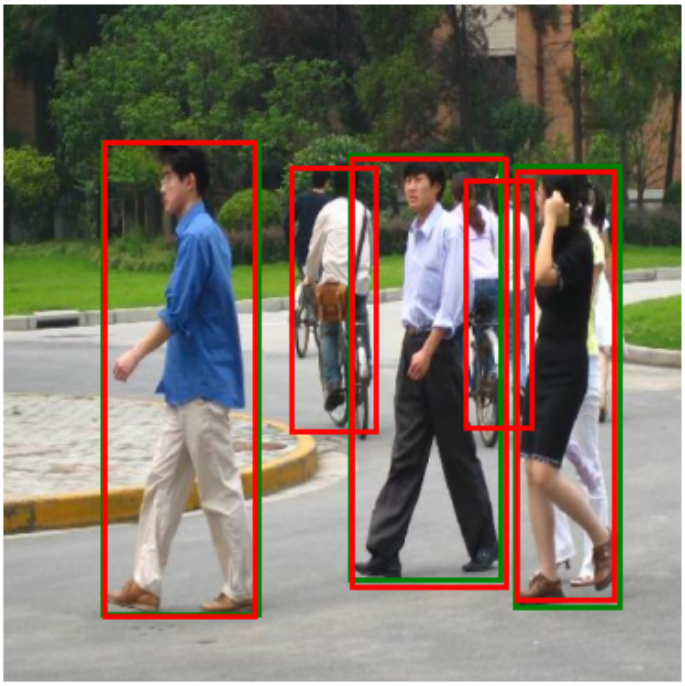

Figure 1: Penn-Fudan RCNN Model Prediction and Ground Truth Boxes

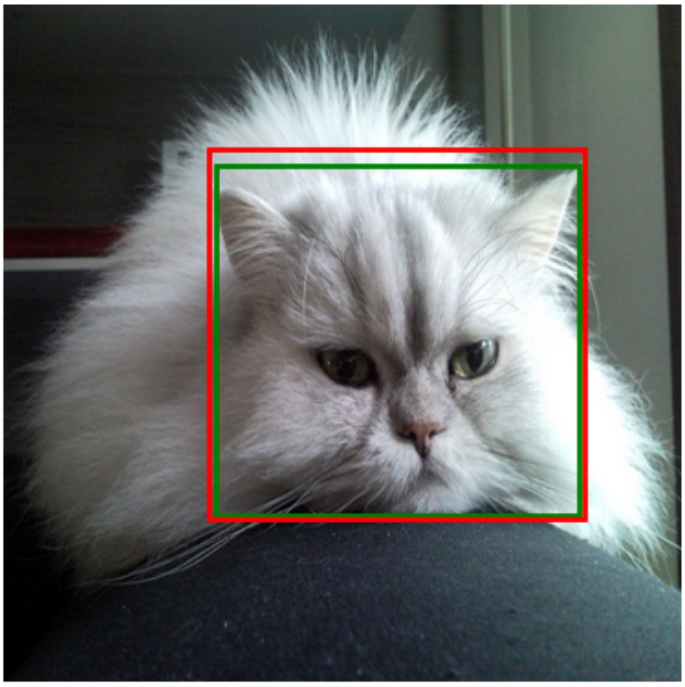

Figure 2: Oxford Pet RCNN Model Prediction and Ground Truth Boxes

6.2.2 YOLO

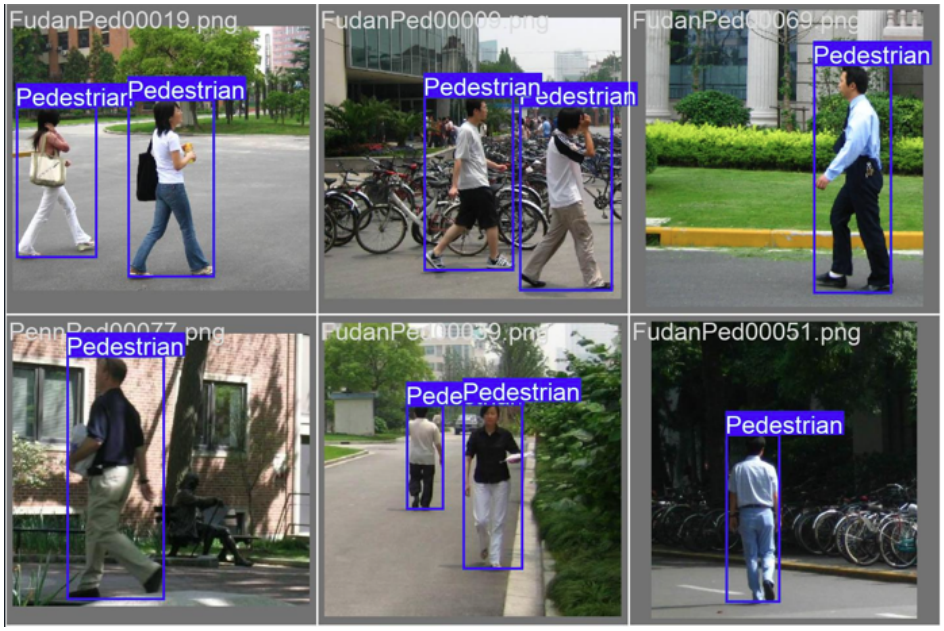

Figure 3: Penn-Fudan YOLO Ground Truth Boxes

Figure 4: Penn-Fudan YOLO Model Predictions

Figure 5: Oxford Pet YOLO Ground Truth Boxes

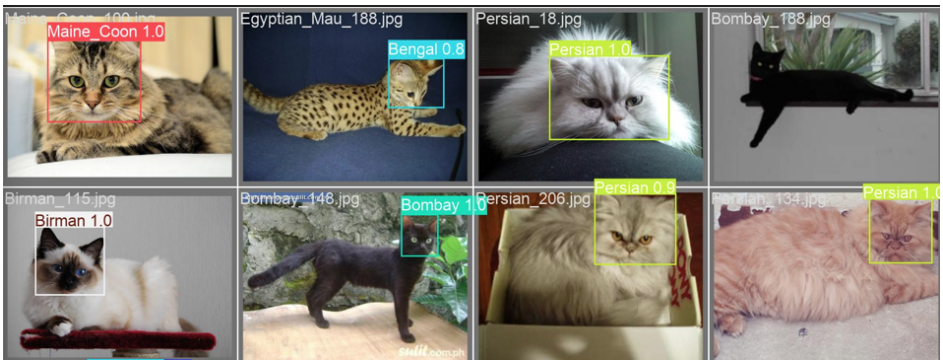

Figure 6: Oxford Pet YOLO Model Predictions

In the RCNN images, the red boxes are the prediction boxes and the green are the ground truth. While in the YOLO images, the top figure is the ground truth and the bottom figure is the model prediction.

From the qualitative results, we can see that the YOLOv8n model consistently outperforms the Faster R-CNN model. YOLO does have occasional issues, such as mislabeling the Egyptian Mau as a Bengal in the Oxford Pet dataset or mistaking a statue for a person in the Penn-Fudan dataset. However, the Faster R-CNN model exhibited more severe problems, frequently generating multiple bounding boxes for a single object and missing some objects entirely. These issues were likely exacerbated by the limited number of training images and the constrained computational resources of an 8GB GPU.

7 Discussion and Conclusion

Overall, the YOLOv8n model demonstrates superior performance for object detection on these small datasets. Despite what the reported inference speeds suggest, both models ran at comparable speeds in practice, but YOLO achieved higher accuracy and more reliable predictions. The mAP50 for YOLO was substantially higher than for Faster R-CNN, and other metrics such as precision and recall were either comparable or slightly lower at worst. Based on these results, we recommend using YOLOv8n for scenarios with limited data and computational resources, as it provides a better balance of accuracy and efficiency compared to Faster R-CNN.